

Дизайн и анализ на алгоритми упражнение 10

Станислав Игов (Стъката)

2025/2026

Най-кратък път. Алгоритъм на Dijkstra. Алгоритъм на Bellman-Ford. Алгоритъм на Floyd-Warshall. Задачи.

Най-кратък път (Shortest Path)

Дефиниция

Нека $G = (V, E, w)$ е претеглен ориентиран или неориентиран граф, където $w : E \rightarrow R$ задава тегло на всяко ребро. **Дължината** на път $p = v_1, v_2, \dots, v_k$ е сумата от теглата на ребрата по пътя:

$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1})$$

Най-кратък път от връх s до връх t е път с минимална дължина сред всички пътища от s до t .

Свойства

- Ако всички тегла са неотрицателни, най-кратък път винаги съществува (ако t е достижим от s).
- Ако има цикъл с отрицателно общо тегло, достижим от s , то най-кратък път може да не съществува (можем да намаляваме дължината безкрайно).
- **Оптимална подструктура:** всеки подпът на най-кратък път също е най-кратък път.

Алгоритъм на Dijkstra

Идея

Алгоритъмът на Dijkstra намира най-кратките пътища от един начален връх s до всички останали върхове в граф с **неотрицателни тегла**. Поддържа масив $dist[1..n]$, където $dist[v]$ е най-краткото известно разстояние от s до v . Използва приоритетна опашка, за да избираме на всяка стъпка върха с най-малко текущо разстояние, който още не е обработен.

Псевдокод

```
Dijkstra(G(V, E, w), s)
@1 dist[1..n] <- {INF..INF}
@2 dist[s] <- 0
@3 visited[1..n] <- {false..false}
@4 PQ <- MinPriorityQueue()
@5 PQ.push(s, 0)
@6
@7 while not PQ.empty do
@8     u <- PQ.pop()
@9     if visited[u] then continue
@10    visited[u] <- true
```

```

@11    forall (v, weight) in adj(u) do
@12        if not visited[v] and dist[u] + weight < dist[v] then
@13            dist[v] <- dist[u] + weight
@14            PQ.push(v, dist[v])
@15        end if
@16    end for
@17 end while
@18 return dist

```

Коректност

На всяка стъпка извличаме от приоритетната опашка върха u с минимално разстояние, който още не е посетен. Тъй като всички тегла са неотрицателни, никой бъдещ път до u не може да бъде по-кратък - всеки такъв път би минал през непосетен връх с по-голямо или равно разстояние и след това би добавил неотрицателни тегла. Следователно в момента на извличане $dist[u]$ е окончателно. С индукция по стъпките: след обработка на k върха, разстоянията до тях са коректни.

Времева сложност

С двоична пирамида (binary heap):

- pop: $O(\log |V|)$, извиква се $O(|V|)$ пъти.
- push: $O(\log |V|)$, извиква се $O(|E|)$ пъти.

$$Time(Dijkstra) \in O((|V| + |E|) \cdot \log |V|)$$

За свързан граф: $O(|E| \cdot \log |V|)$.

Важно: Алгоритъмът на Dijkstra работи коректно само за графи с неотрицателни тегла.

Алгоритъм на Bellman-Ford

Алгоритъмът на Bellman-Ford също намира най-кратките пътища от един начален връх, но работи и с **отрицателни тегла**. Извършва $|V| - 1$ итерации, като на всяка итерация обхожда всички ребра и релаксира разстоянията. Ако след $(|V| - 1)$ -тата итерация може да се направи още релаксация, значи в графа има отрицателен цикъл.

$$Time(BellmanFord) \in O(|V| \cdot |E|)$$

Алгоритъм на Floyd-Warshall

Алгоритъмът на Floyd-Warshall намира най-кратките пътища между **всички двойки** върхове. Използва динамично програмиране: на стъпка k разглеждаме пътища, които минават само през върхове $\{1, 2, \dots, k\}$ като междинни.

$$Time(FloydWarshall) \in O(|V|^3)$$

Задачи с най-къси пътища

Задача 1: Авиокомпания - максимален брой пътници

Условие: Авиокомпания ФлайКо извършва полети между n града, номерирани от 1 до n . Има m двупосочни полета. За всеки полет са известни двата града, които свързва, и максималният брой пътници, които самолетът може да превози. Тръгвайки от град 1, искаме да намерим максималния брой пътници, които могат да достигнат до всеки от останалите градове, използвайки само един маршрут (път без повторения).

Ограничението по маршрута е следното: капацитетът на маршрут се определя от полета с най-малък капацитет. Търсим маршрут, който максимизира тази минимална стойност.

Вход: Число n (брой градове) и m (брой полети). Масив от m ребра, всяко задава два града и капацитет.

Изход: За всеки град от 2 до n - максималният брой пътници, които могат да достигнат до него от град 1.

Идея

Това е модификация на алгоритъма на Dijkstra. Вместо да минимизираме сума от тегла, максимизираме минималния капацитет по пътя.

Стъпки

1. Инициализираме масив $cap[1..n]$, където $cap[v]$ ще пази максималния капацитет на маршрут от връх 1 до връх v . Задаваме $cap[1] = \infty$ (от началния връх до себе си няма ограничение) и $cap[v] = 0$ за всички останали.
2. Използваме приоритетна опашка по максимум (max-heap). Поставяме в нея връх 1 с приоритет ∞ .
3. На всяка стъпка извличаме от опашката върха u с най-голям капацитет, който още не е посетен. Маркираме го като посетен.
4. За всеки съсед v на u с капацитет на полета w : изчисляваме $\min(cap[u], w)$. Ако тази стойност е по-голяма от текущия $cap[v]$, обновяваме $cap[v]$ и добавяме v в опашката с новия приоритет.
5. След приключване на алгоритъма, $cap[v]$ е отговорът за всеки град v .

Коректност

Задачата е аналогична на задачата за плаване на кораб от упражнение 9, но тук търсим максимин път от конкретен начален връх до всички останали.

Когато извлечем връх u от max-heap-а, $cap[u]$ е окончателен. Защо? Всеки алтернативен път до u би минал през непосетен връх v с $cap[v] \leq cap[u]$ (защото u е с най-голям капацитет в опашката). От v нататък капацитетът може само да намалее (взимаме минимум), следователно алтернативният път не може да даде по-голям капацитет. Това е аналогично на аргумента за коректност на Dijkstra, но вместо “сумата расте” използваме “минимумът намалява”.

Времева сложност

$$Time \in O((n + m) \cdot \log n)$$

Същата като стандартния Dijkstra - всяко ребро предизвиква най-много една операция с приоритетната опашка.

Задача 2: Ванката бърза

Условие: Ванката иска да стигне от дома си (връх 1) до офиса (връх n) по възможно най-бърз начин. Градът е представен като свързан граф с n кръстовища и m двупосочни улици. Ванката тръгва с Голф и по всяко време може да спре Голфа на някое кръстовище и да продължи с колело. Веднъж слязъл от Голфа, не може да се върне на него. За всяка улица са дадени две времена: a_i (с Голфа) и b_i (с колело).

Вход: Число n (брой кръстовища) и m (брой улици). Масив от m ребра, всяко задава два кръстовища и две времена (Голф и колело).

Изход: Минималното време от връх 1 до връх n .

Идея

Идеята е да дублираме графа, за да моделираме двете “фази” на пътуването: фаза 1 (с Голф) и фаза 2 (с колело).

Стъпки

1. Построяваме нов граф с $2n$ върха. Върховете $1, 2, \dots, n$ представят кръстовищата, когато Ванката е с Голфа. Върховете $n + 1, n + 2, \dots, 2n$ представят същите кръстовища, но когато Ванката е с колело.
2. За всяко ребро (u, v) от оригиналния граф добавяме:
 - Ребро (u, v) с тегло a_i (движение с Голфа в горния слой).
 - Ребро $(n + u, n + v)$ с тегло b_i (движение с колело в долния слой).
3. За всеки връх v от 1 до n добавяме еднопосочно ребро от v до $n + v$ с тегло 0. Това моделира момента, в който Ванката оставя Голфа и взема колелото. Тези ребра са само надолу (от горния слой към долния), така че Ванката не може да се върне на Голфа.
4. Пускаме алгоритъма на Dijkstra от връх 1 (тръгва с Голфа) върху новия граф с $2n$ върха.
5. Отговорът е $dist[2n]$ - защото ние имаме ребро от n до $2n$ с тегло 0. Тоест ако Ванката стигне до офиса (n) само с кола, то за същото време ще стигне до $(2n)$.

Коректност

Всеки път в новия граф съответства на валидно пътуване на Ванката. В горния слой Ванката е с Голфа, в долния - с колело. Еднопосочните ребра с тегло 0 от горния към долния слой означават, че Ванката може да слезе от Голфа на кое да е кръстовище, но след като премине в долния слой, не може да се върне (няма ребра от долния към горния слой). Следователно преходът Голф $\rightarrow$ колело се случва най-много веднъж, което отговаря на условието. Всички тегла са положителни, така че Dijkstra работи коректно. $dist[2n]$ покрива и двата случая: стигане само с Голф или смяна на колело по пътя.

Времева сложност

Новият граф има $2n$ върха и $2m + n$ ребра. Dijkstra работи в:

$$Time \in O((2n + 2m + n) \cdot \log(2n)) = O((n + m) \cdot \log n)$$

Задача 3: Най-близкото важно място

Условие: Град е представен като граф с n места, номерирани от 1 до n , и m двупосочни транспортни връзки с дадено време за преминаване. Определени са k важни места. За всеки от q въпроса се дава номер на хотел и трябва да се отговори какво е минималното разстояние от този хотел до най-близкото важно място.

Вход: Числа n и m . Масив от m ребра, всяко задава два върха и време. Число k и масив от k важни места. Число q и масив от q заявки (номера на хотели).

Изход: За всяка заявка - разстоянието до най-близкото важно място.

Идея

Вместо да пускаме Dijkstra от всеки хотел поотделно (което би било бавно), добавяме нов виртуален връх и пускаме Dijkstra само веднъж.

Стъпки

1. Добавяме нов виртуален връх 0 към графа.
2. Свързваме връх 0 с всяко от k -те важни места чрез еднопосочно ребро с тегло 0 (от 0 към важното място).
3. Пускаме алгоритъма на Dijkstra от връх 0 върху разширения граф.
4. След изпълнението, $dist[v]$ за всеки връх v е минималното разстояние от v до най-близкото важно място.
5. За всяка заявка с хотел h отговорът е $dist[h]$.

Коректност

Виртуалният връх 0 е свързан с тегло 0 до всички важни места. Следователно най-краткият път от 0 до произволен връх v минава през точно едно важно място - това, което е най-близко до v . Теглото 0 на ребрата от 0 до важните места гарантира, че $dist[v]$ е точно разстоянието от v до най-близкото важно място. Това е еквивалентно на пускане на Dijkstra от всички важни места едновременно (multi-source Dijkstra), но моделирано чрез един виртуален начален връх.

Времева сложност

Добавяме k ребра, така че новият граф има $n + 1$ върха и $m + k$ ребра:

$$Time \in O((n + m + k) \cdot \log n)$$

Тъй като $k \leq n$, това е $O((n + m) \cdot \log n)$. Отговаряме на всички q заявки за $O(q)$.

Задача 4: Крайни резултати в игра (FiniteScores)

Условие: Игра $\mathcal{G} = (P, \delta, s)$ се определя от:

- Крайно множество от позиции P , номерирани от 1 до $|P|$.
- Частична функция $\delta : P \times \{\ell, r, u, d\} \rightarrow P \times Z$, която по дадена позиция и посока дава нова позиция и точки за хода.
- Начална позиция $s \in P$.

Епизод е дума $\alpha = a_1 \dots a_n$, с която от начална позиция s се стига до позиция без ходове. Резултатът от епизод е $score(\alpha) = \prod_{i=1}^n 2^{c_i}$, където c_i са точките от всеки ход. Задачата е да се определи дали съществува число $b \in Q$, такова че $|score(\alpha)| \leq b$ за всеки епизод α .

Вход: Множество от позиции P с $|P|$ елемента, списък от преходи (позиция, посока, нова позиция, точки), начална позиция s .

Изход: “Да”, ако резултатите на всички епизоди са ограничени. “Не” иначе.

Идея

Първо забелязваме ключова връзка:

$$score(\alpha) = \prod_{i=1}^n 2^{c_i} = 2^{\sum_{i=1}^n c_i}$$

Следователно $|score(\alpha)| \leq b$ за някое b тогава и само тогава, когато сумата $\sum c_i$ е ограничена отгоре за всички епизоди. Ако има епизод с произволно голяма сума, то $score$ расте неограничено.

Сумата може да расте неограничено тогава и само тогава, когато в графа на играта (разглеждан като ориентиран граф с тегла c_i по ребрата) съществува цикъл с положителна сума на теглата, достижим от s .

Стъпки

1. Построяваме ориентиран граф, в който върховете са позициите от P . За всеки преход $\delta(p, a) = (q, c)$ добавяме ребро от p до q с тегло c . Забележка: всяка позиция има най-много 4 изходящи ребра (по едно за всяка посока), така че общият брой ребра е най-много $4|P|$.
2. Обръщаме знака на всички тегла: ребро с тегло c става ребро с тегло $-c$. Така положителен цикъл в оригиналния граф се превръща в отрицателен цикъл.
3. Пускаме алгоритъма на Bellman-Ford от началната позиция s върху графа с обрънатите тегла.
4. Ако Bellman-Ford открие отрицателен цикъл (т.е. на $|P|$ -тата итерация все още може да се направи релаксация), отговаряме “Не” - резултатите не са ограничени, защото има цикъл с положителна сума на точките.
5. Ако Bellman-Ford не открие отрицателен цикъл, отговаряме “Да” - всички резултати са ограничени.

Коректност

Резултатът $score(\alpha) = 2^{\sum c_i}$ е ограничен за всички епизоди тогава и само тогава, когато сумата $\sum c_i$ е ограничена отгоре. Ако съществува цикъл с положителна сума на точките, достигим от s , можем да обходим този цикъл произволен брой пъти и да направим сумата произволно голяма, следователно и $score$ произволно голям. Обратно, ако няма такъв цикъл, всеки епизод минава през ацикличен път (ограничен на дължина $|P|$), и сумата е ограничена.

Обръщането на знаците ни позволява да сведем задачата до търсене на отрицателен цикъл, което е стандартна задача за Bellman-Ford. Алгоритъмът на Bellman-Ford извършва $|P| - 1$ итерации на релаксация по всички ребра, което е достатъчно да намери най-кратките пътища, ако няма отрицателен цикъл. Ако на допълнителна $|P|$ -та итерация все още има релаксация, значи има отрицателен цикъл.

Времева сложност

Bellman-Ford работи в $O(|V| \cdot |E|)$. Тук $|V| = |P|$ и $|E| \leq 4|P|$, следователно:

$$Time \in O(|P| \cdot 4|P|) = O(|P|^2)$$

Кое е точно търсената сложност $O(|P|^2)$.